

Follow these steps to install MySQL and create a user:

Step 1: Update Package Index

Step 4: Create a User 'harry'@'localhost'

sudo apt update

CREATE USER 'harry'@'localhost' IDENTIFIED BY 'password';

GRANT ALL PRIVILEGES ON *.* TO 'harry'@'localhost' WITH GRANT OPTION;

FLUSH PRIVILEGES;

Step 1: Create a Database

Right-click it in MySQL Workbench and select "Set as Default Schema", or

Use this SQL command:

CREATE DATABASE startersql;

USE startersql;

Step 2: Create a Table

Now we'll create a simple users table:

Step 3: Drop the Database

You can delete the entire database (and all its tables) using:

Be careful – this will delete everything in that database.

CREATE TABLE users (

 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

DROP DATABASE startersql;

when a row is created.

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP , is_active BOOLEAN

Selecting Data from a Table

Select All Columns

Select Specific Columns

To rename an existing table:

To rename it back:

SELECT * FROM users;

SELECT name, email FROM users;

RENAME TABLE users TO customers;

RENAME TABLE customers TO users;

Altering a Table

You can use ALTER TABLE to modify an existing table.

Drop a Column

ALTER TABLE users ADD COLUMN is_active BOOLEAN DEFAULT TRUE;

ALTER TABLE users DROP COLUMN is_active;

ALTER TABLE users MODIFY COLUMN name VARCHAR(150);

ALTER TABLE users MODIFY COLUMN email VARCHAR(100) FIRST;

ALTER TABLE users MODIFY COLUMN gender ENUM('Male', 'Female', 'Other') AFTER name;

Inserting Data into MySQL Tables

To add data into a table, we use the INSERT INTO statement.

Insert Without Specifying Column Names (Full Row

Insert)

Insert by Specifying Column Names (Best Practice)

This method is safer and more readable. You only insert into specific columns.

INSERT INTO users VALUES

INSERT INTO users (name, email, gender, date_of_birth) VALUES

INSERT INTO users (name, email, gender, date_of_birth) VALUES

The remaining columns like id (which is AUTO_INCREMENT) and created_at

Insert Multiple Rows at Once

This is more efficient than inserting rows one by one.

INSERT INTO users (name, email, gender, date_of_birth) VALUES

Querying Data in MySQL using SELECT

The SELECT statement is used to query data from a table.

To select all columns:

SELECT column1, column2 FROM table_name;

SELECT * FROM users;

SELECT * FROM users WHERE gender = 'Male';

SELECT * FROM users WHERE gender != 'Female';

SELECT * FROM users WHERE gender <> 'Female';

SELECT * FROM users WHERE date_of_birth < '1995-01-01';

SELECT * FROM users WHERE id > 10;

SELECT * FROM users WHERE id >= 5;

SELECT * FROM users WHERE id <= 20;

SELECT * FROM users WHERE date_of_birth IS NULL;

SELECT * FROM users WHERE date_of_birth IS NOT NULL;

SELECT * FROM users WHERE date_of_birth BETWEEN '1990-01-01' AND '2000-12-31';

SELECT * FROM users WHERE gender IN ('Male', 'Other');

SELECT * FROM users WHERE name LIKE 'A%';

SELECT * FROM users WHERE name LIKE '%a';

SELECT * FROM users WHERE name LIKE '%li%'; -- Contains 'li'

SELECT * FROM users WHERE gender = 'Female' AND date_of_birth > '1990-01-01';

SELECT * FROM users WHERE gender = 'Male' OR gender = 'Other';

SELECT * FROM users ORDER BY date_of_birth ASC;

SELECT * FROM users ORDER BY name DESC;

SELECT * FROM users LIMIT 5;

SELECT * FROM users LIMIT 10 OFFSET 5;

```

SELECT * FROM users LIMIT 5, 10;
SELECT * FROM users ORDER BY created_at DESC LIMIT 10;
SELECT * FROM users WHERE salary > 60000 ORDER BY created_at DESC LIMIT 5;
SELECT * FROM users ORDER BY salary DESC;
SELECT * FROM users WHERE salary BETWEEN 50000 AND 70000;
UPDATE - Modifying Existing Data
The UPDATE statement is used to change values in one or more rows.
Example: Update One Column
Example: Update Multiple Columns
Without WHERE Clause (Warning)
UPDATE table_name
UPDATE users
UPDATE users
UPDATE users
This updates every row in the table. Be very careful when omitting the WHERE
Quick Quiz: Practice Your UPDATE Skills
1. Update the salary of user with id = 5 to ■70,000.
UPDATE users
UPDATE users
UPDATE users
Note: This query will overwrite salary for every user. Use with caution!
UPDATE users
UPDATE users
DELETE - Removing Data from a Table
The DELETE statement removes rows from a table.
Example: Delete One Row
Delete Multiple Rows
Delete All Rows (but keep table structure)
Drop the Entire Table (use with caution)
DELETE FROM table_name
DELETE FROM users
DELETE FROM users
DELETE FROM users;
DROP TABLE users;
Always use WHERE unless you're intentionally updating/deleting everything.
Consider running a SELECT with the same WHERE clause first to confirm what
Quick Quiz: Practice Your DELETE Skills
SELECT * FROM users WHERE id = 3;
DELETE FROM users
DELETE FROM users
Add UNIQUE using ALTER TABLE :
CREATE TABLE users (
ALTER TABLE users
Naming the constraint ( chk_dob ) helps if you want to drop it later.
Sets a default value for a column if none is provided during insert.
CREATE TABLE users (
ALTER TABLE users
ALTER TABLE users
ALTER TABLE users
Add DEFAULT using ALTER TABLE :
Add later with ALTER TABLE :
CREATE TABLE users (
ALTER TABLE users
ALTER COLUMN is_active SET DEFAULT TRUE;
CREATE TABLE users (
ALTER TABLE users
CREATE TABLE users (
We'll use the users table which includes:
id , name , email , gender , date_of_birth , salary , created_at
SELECT COUNT(*) FROM users;
SELECT COUNT(*) FROM users WHERE gender = 'Female';
SELECT MIN(salary) AS min_salary, MAX(salary) AS max_salary FROM users;
SELECT SUM(salary) AS total_payroll FROM users;
SELECT AVG(salary) AS avg_salary FROM users;
SELECT gender, AVG(salary) AS avg_salary
SELECT name, LENGTH(name) AS name_length FROM users;
SELECT name, LOWER(name) AS lowercase_name FROM users;
SELECT name, UPPER(name) AS uppercase_name FROM users;
SELECT CONCAT(name, ' <', email, '>') AS user_contact FROM users;
SELECT NOW();
SELECT name, YEAR(date_of_birth) AS birth_year FROM users;
SELECT name, DATEDIFF(CURDATE(), date_of_birth) AS days_lived FROM users;
SELECT name, TIMEDIFF(YEAR, date_of_birth, CURDATE()) AS age FROM users;
SELECT salary,
SELECT id, MOD(id, 2) AS remainder FROM users;
SELECT name, gender,
MySQL Transactions and AutoCommit

```

By default, MySQL operates in AutoCommit mode. This means that every SQL statement is treated as a transaction and is committed automatically. However, for more control over when changes are saved, you can turn AutoCommit off and

1. Disabling AutoCommit

When AutoCommit is off, you can explicitly control when to commit or rollback

To disable AutoCommit:

This turns off AutoCommit, meaning that changes you make won't be saved to the database unless you explicitly tell MySQL to commit them.

Important: Until you execute a COMMIT, your changes are not permanent.

2. COMMIT – Save Changes to the Database

use the COMMIT command to save those changes.

To commit a transaction:

```
SET autocommit = 0;
```

```
COMMIT;
```

This saves all the changes made since the last COMMIT or ROLLBACK. After this

3. ROLLBACK – Revert Changes to the Last Safe Point

rollback the transaction to its previous state.

To rollback a transaction:

This undoes all changes since the last COMMIT or ROLLBACK.

Here's a simple example of using COMMIT and ROLLBACK in a transaction:

Turn off AutoCommit:

Decide whether to commit or rollback:

```
ROLLBACK;
```

```
SET autocommit = 0;
```

```
UPDATE users SET salary = 80000 WHERE id = 5;
```

```
COMMIT;
```

4. Enabling AutoCommit Again

If you want to turn AutoCommit back on (so that every statement is automatically committed), you can do so with:

Use COMMIT when you want to make changes permanent.

Use ROLLBACK to discard changes if something goes wrong.

Consider disabling AutoCommit when performing complex updates to avoid

```
ROLLBACK;
```

```
SET autocommit = 1;
```

```
CREATE TABLE users (
```

Dropping

Cannot be easily dropped

Can be dropped anytime

Can I Drop a PRIMARY KEY?

Yes, but it is more restricted than dropping a UNIQUE constraint.

```
CREATE TABLE users (
```

```
ALTER TABLE users DROP PRIMARY KEY;
```

To drop a UNIQUE constraint:

This means that every time you insert a new row, MySQL will automatically assign a

Use PRIMARY KEY for the main identifier of a row.

Use UNIQUE for enforcing non-duplicate values in other columns (like email

```
ALTER TABLE users DROP INDEX email;
```

```
CREATE TABLE users (
```

```
ALTER TABLE users AUTO_INCREMENT = 1000;
```

A foreign key is a column that creates a link between two tables. It ensures that

Why Use Foreign Keys?

putting address columns inside the users table, you create a separate

Let's create an addresses table where each address belongs to a user.

```
CREATE TABLE addresses (
```

Dropping a Foreign Key

To drop a foreign key, you need to know its constraint name. MySQL auto-

To drop it:

Adding a Foreign Key Later (Using ALTER)

later using ALTER TABLE :

```
CREATE TABLE addresses (
```

```
ALTER TABLE addresses
```

```
DROP FOREIGN KEY fk_user;
```

```
ALTER TABLE addresses
```

Adding ON DELETE Action

By default, if you delete a user that has related addresses, MySQL will throw an

error. You can control this behavior with ON DELETE.

Example with ON DELETE CASCADE :

If you want addresses to be automatically deleted when the user is deleted:

Or alter it later:

Other ON DELETE Options

```
ON DELETE
```

Deletes all related rows in child table

```
CREATE TABLE addresses (
```

```
CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

```
ALTER TABLE addresses
```

```
ADD CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE
```

You can create them inline or with ALTER TABLE.

You can drop them by name.
 Use ON DELETE to control what happens when the parent row is deleted.
 We'll use the following two tables:
 Raj is excluded because there is no matching address. Delhi is excluded because its user_id (4) is not in users .
 SELECT users.name, addresses.city
 SELECT users.name, addresses.city
 SELECT users.name, addresses.city
 SELECT statements. It removes duplicates by default.
 If you want to include all rows including duplicates, use UNION ALL .
 You already have a users table for active users. Now, we'll create an
 Step 1: Create the admin_users Table
 CREATE TABLE admin_users (
 Step 2: Insert Sample Data into admin_users
 Step 3: Use UNION to Combine Data
 If you want to keep duplicate names (if any), use UNION ALL .
 You can also select multiple columns as long as both SELECT queries return the
 INSERT INTO admin_users (id, name, email, gender, date_of_birth, salary) VALUES
 SELECT name FROM users
 SELECT name FROM admin_users;
 SELECT name FROM users
 SELECT name FROM admin_users;
 The number of columns and their data types must match in all SELECT
 SELECT name, salary FROM users
 SELECT name, salary FROM admin_users;
 SELECT name, 'User' AS role FROM users
 SELECT name, 'Admin' AS role FROM admin_users;
 SELECT name FROM users
 SELECT name FROM admin_users
 When to Use UNION
 Step 2: Insert Referral Data (Optional)
 ALTER TABLE users
 UPDATE users SET referred_by_id = 1 WHERE id IN (2, 3); -- User 1 referred Users 2
 UPDATE users SET referred_by_id = 2 WHERE id = 4;
 Step 3: Use a Self JOIN to Get Referrer Names
 Use Self JOIN when you need to join a table with itself.
 SELECT
 Use aliases like a and b to differentiate the two instances of the same table.
 A view in MySQL is a virtual table based on the result of a SELECT query. It does
 You want to reuse logic
 CREATE VIEW high_salary_users AS
 SELECT id, name, salary
 SELECT * FROM high_salary_users;
 Step 1: View before update
 Step 2: Update a user's salary
 itself. That's because views always reflect live data from the original table.
 SELECT * FROM high_salary_users;
 UPDATE users
 SELECT * FROM high_salary_users;
 Dropping a View
 Views act like saved SELECT queries
 DROP VIEW high_salary_users;
 automatically created primary key index.
 Creates an index named idx_email
 SHOW INDEXES FROM users;
 CREATE INDEX idx_email ON users(email);
 SELECT * FROM users WHERE email = 'example@example.com';
 Indexes slow down INSERT , UPDATE , and DELETE operations slightly
 (because the index must be updated)
 Use indexes only when needed (i.e., for columns used in WHERE , JOIN ,
 CREATE INDEX idx_gender_salary ON users(gender, salary);
 SELECT * FROM users
 But this may not use the index effectively:
 Because the first column in the index (gender) is missing in the filter.
 Dropping an Index
 To delete an index:
 SHOW INDEXES
 CREATE INDEX
 Create single or multi-column indexes
 DROP INDEX
 Use when
 DROP INDEX idx_email ON users;
 SELECT statements
 The inner query: SELECT AVG(salary) FROM users returns the average salary.
 SELECT id, name, salary
 SELECT AVG(salary) FROM users
 The outer query selects all users with a salary greater than that average.

The inner query: `SELECT id FROM users WHERE salary > 75000` returns a list
The outer query selects users whose `referred_by_id` is in that list.
You can also use subqueries:
Inside `SELECT` columns (called scalar subqueries)
In the `FROM` clause to create derived tables
Example in `SELECT` :
`SELECT id, name, referred_by_id`
`SELECT id FROM users WHERE salary > 75000`
`SELECT name, salary,`
`(SELECT AVG(salary) FROM users) AS average_salary`
Use Case
Subquery in `SELECT`
The `GROUP BY` clause is used to group rows that have the same values in
The `HAVING` clause is used to filter groups after aggregation – similar to how
`SELECT gender, AVG(salary) AS average_salary`
Let's say we only want to show genders where the average salary is greater than
`SELECT referred_by_id, COUNT(*) AS total_referred`
`SELECT gender, AVG(salary) AS avg_salary`
Use `GROUP BY` to organize data, and `HAVING` to filter those groups based on
To get subtotals and grand totals, you can use `ROLLUP` :
`SELECT referred_by_id, COUNT(*) AS total_referred`
`SELECT gender, COUNT(*) AS total_users`
queries, updates, or conditional operations.
Why Change the Delimiter?
But when defining stored procedures, we use `;` inside the procedure as well. This
can confuse MySQL. To avoid this, we temporarily change the delimiter (e.g. to
After the procedure is created, we reset the delimiter back to `;` .
`DELIMITER $$`
`CREATE PROCEDURE procedure_name()`
`DELIMITER ;`
Let's say you want to create a stored procedure that inserts a new user into the
This creates a procedure named `AddUser` that accepts five input parameters.
You can call the procedure using:
This will insert the new user into the `users` table.
`DELIMITER $$`
`CREATE PROCEDURE AddUser(`
`INSERT INTO users (name, email, gender, date_of_birth, salary)`
`DELIMITER ;`
`CALL AddUser('Kiran Sharma', 'kiran@example.com', 'Female', '1994-06-15', 72000);`
Dropping a Stored Procedure
`DELIMITER $$`
Temporarily change statement delimiter
`CREATE PROCEDURE`
`CALL procedure_name(...)`
`DROP PROCEDURE`
`SHOW PROCEDURE STATUS WHERE Db = 'startersql';`
`DROP PROCEDURE IF EXISTS AddUser;`
(triggered) when a specific event occurs in a table – such as `INSERT` , `UPDATE` , or
`DELETE` .
On `INSERT` , `UPDATE` , or `DELETE`
Scenario: Log Every New User Insertion
Suppose we want to log every time a new user is inserted into the `users` table.
We'll create a separate table called `user_log` to store log entries.
`CREATE TRIGGER trigger_name`
`AFTER INSERT ON table_name`
Step 1: Create the Log Table
Step 2: Create the Trigger
`AFTER INSERT` means the trigger fires after the user is inserted.
We insert the new user's ID and name into the `user_log` table.
`CREATE TABLE user_log (`
`created_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP`
`DELIMITER $$`
`CREATE TRIGGER after_user_insert`
`AFTER INSERT ON users`
`INSERT INTO user_log (user_id, name)`
`DELIMITER ;`
Dropping a Trigger
`INSERT / UPDATE /`
`DELETE`
Refers to the new row (for `INSERT` , `UPDATE`)
Refers to the old row (for `UPDATE` , `DELETE`)
`CALL AddUser('Ritika Jain', 'ritika@example.com', 'Female', '1996-03-12', 74000);`
`SELECT * FROM user_log;`
`DROP TRIGGER IF EXISTS after_user_insert;`
Use `ALTER TABLE` to add a column:
`ALTER TABLE users`
Alternative syntax:

```
SELECT * FROM users
```

```
SELECT * FROM users
```

6. TRUNCATE Keyword

TRUNCATE removes all rows from a table, but keeps the table structure.

Faster than DELETE FROM users

Both CHANGE and MODIFY are used to alter existing columns in a table, but they

CHANGE: Rename and change datatype

This renames city to location and changes its type.

```
SELECT DISTINCT gender FROM users;
```

```
TRUNCATE TABLE users;
```

```
ALTER TABLE users
```

```
ALTER TABLE users
```